CAMPUSNODE LMS

Software Engineering Final Project Report

Course: CS4433 – Software Engineering
Project: CampusNode Learning Management System
Repository: https://github.com/King-Eazii/CampusNode-LMS
Technology: C++17, SQLite, CMake and libsodium
Release: v1.0 (release target)

Abstract

CampusNode LMS is a C++17 console-based Learning Management System designed to support the management of academic activities within a school. The system provides facilities for managing user accounts, departments, courses, student enrolments, assignments, submissions and assessment results.

The application supports four user roles: Administrator, Lecturer, Head of Department (HoD) and Student. It uses SQLite for persistent local storage and libsodium for secure password hashing using Argon2id. The system is designed to operate offline on a single school computer.

This report presents the problem addressed by the system, its requirements, use cases, proposed class-based solution, architecture, design, implementation and testing. Eighteen automated unit and integration tests passed during local testing. However, formal system testing, measured code coverage and confirmation of the hosted continuous integration workflow remain outstanding.

1. Problem Identified and Proposed Solution

Schools require an organised way to manage academic activities, including student records, courses, assignments, submissions, marks and feedback. When these activities are managed manually or through separate records, information can become inconsistent, duplicated or difficult to access. There is also a risk of unauthorised access to sensitive academic information.

CampusNode LMS addresses these challenges through a centralised application that allows authorised users to perform their academic responsibilities within one system.

The proposed solution is a locally installed learning management system that operates without requiring an internet connection. Administrators manage users and academic records, lecturers manage assignments and marking, HoDs moderate and release results, and students access their courses, submit assignments and view released marks.

SQLite was selected to provide persistent local storage, while C++17 was used to implement the application. This approach is suitable for the initial requirement of operating on one school computer without the complexity of maintaining a network server.

2. Functional and Non-Functional Requirements

2.1 Functional Requirements

FR-01: Academic Administration

The system shall allow an Administrator to create and manage user accounts, departments and courses, as well as enrol students in courses.

FR-02: Assignment and Assessment Management

The system shall allow Lecturers to create assignments, receive student submissions, assign marks and provide feedback. HoDs shall be able to review, modify marks with a recorded reason and release results.

2.2 Non-Functional Requirements

NFR-01: Security

The system shall protect user accounts through secure password hashing and role-based access control. Users shall only be permitted to perform actions authorised for their roles.

NFR-02: Persistence and Offline Operation

The system shall store academic information persistently in a local SQLite database and support normal operation without an internet connection.

2.3 Important Business Rules

The system implements the following assessment rules:

- Late submissions receive a penalty of 10% of the maximum mark per day or part of a day, up to three days.
- Submissions received more than three days late receive zero marks.
- Students may resubmit before the effective deadline, including any approved extension.
- Earlier submission versions are retained.
- Students may only view marks and feedback after the HoD releases them.
- Extensions require a written reason and must be recorded.

3. Use Cases

3.1 Use Case: Manage Academic Records

Primary actor: Administrator.

The Administrator logs into the system and manages user accounts, departments and courses. The Administrator can also enrol students in courses.

The system validates the information, checks the Administrator's permissions and stores the records in the SQLite database.

Expected outcome: Academic records and enrolments are created and made available to authorised users.

3.2 Use Case: Create and Assess an Assignment

Primary actor: Lecturer.

The Lecturer creates an assignment and specifies its deadline. Students enrolled in the relevant course can submit their work before the deadline. They may also resubmit before the effective deadline.

The Lecturer views submissions, assigns marks and provides feedback. The HoD then reviews the marks, performs moderation where necessary and releases the results.

Expected outcome: Assignment submissions are assessed, moderated and released, after which students can access their marks and feedback.

4. Proposed Solution in Class Format

CampusNode LMS uses an object-oriented design to represent the different entities and responsibilities within the system.

The main classes include:

- User: Abstract base class representing a system user.
- Administrator: Handles user and academic record management.
- Lecturer: Handles assignments, submissions, marking and feedback.
- HoD: Handles moderation and release of departmental results.
- Student: Represents students who access courses and submit assignments.
- Department: Represents an academic department.
- Course: Represents a course offered by a department.
- Enrolment: Connects students to courses.
- Assignment: Represents an assessment created by a lecturer.
- Submission: Stores submitted assignment versions.
- Assessment/Mark: Stores marks and feedback.
- AuditLog: Records important system activities, including mark changes.

The Administrator, Lecturer, HoD and Student classes inherit from the abstract User class. This supports inheritance and polymorphism by allowing different user types to share common account functionality while maintaining role-specific operations.

The Course class is associated with a Department, while students are connected to courses through Enrolment. Assignments belong to courses, and submissions are associated with both students and assignments.

The design separates business logic from database operations. Service classes handle application rules, while repository and data-access classes communicate with SQLite. This separation improves maintainability and makes business rules easier to test.

5. System Architecture and Design

5.1 System Architecture

CampusNode LMS follows a layered architecture and is deployed on a single school computer.

The architecture consists of:

Presentation Layer: Provides the console interface through which users interact with the application.

Business Logic Layer: Processes application operations, validates information, enforces role permissions and applies assessment rules.

Data Access Layer: Handles communication between the application and the SQLite database using parameterised queries.

Database Layer: Stores user accounts, departments, courses, enrolments, assignments, submissions, marks and audit records.

Supporting Libraries: SQLite provides database functionality, while libsodium provides password-hashing functionality.

The application does not require a network connection for normal operation. The database is stored locally and is not intended to be shared directly across computers through a network folder.

5.2 Security and Database Design

The system uses SQLite to maintain persistent academic records. Database access is handled through the data-access layer, and parameterised queries are used to reduce the risk of SQL injection.

Passwords are not stored in plain text. Instead, libsodium implements Argon2id password hashing using randomly generated salts.

The implementation also includes:

- Role-based access control.
- Login protection after five incorrect attempts, with the username locked until the program restarts.
- Database foreign-key enforcement.
- Audit-log protection against updates and deletions through database triggers.
- Database backup using SQLite VACUUM INTO.
- Database validation during restoration.

The submission process follows the lifecycle:

Submitted/Late → Marked → Moderated → Released

Earlier submission versions are retained when students resubmit work.

6. Implementation Process

CampusNode LMS was implemented using C++17 and built using CMake. SQLite was integrated for persistent storage, while libsodium was used for password hashing.

The implementation separates application services from database operations. Service classes coordinate system activities and enforce business rules, while repository components handle database queries and persistence.

During first execution, the application creates the database and required tables. The first Administrator account can then be created. The system does not use a predefined default administrator password.

The application executable in the Windows development environment is:

```
build\Debug\lms.exe
```

Git was used for source-code version control, and the project repository is hosted on GitHub:

https://github.com/King-Eazii/CampusNode-LMS

A GitHub Actions workflow was created to automate project configuration, compilation and test execution when changes are pushed or pull requests are made. However, a successful hosted workflow execution has not yet been confirmed.

7. Testing and Verification

Testing was conducted using GoogleTest, with CTest used to execute the automated tests.

The project contains 18 automated unit and integration tests in `tests/BusinessLogicTests.cpp`.

7.1 Unit Testing

Unit tests cover:

- Late-penalty calculations.
- Mark validation.
- Username validation.
- Required-text validation.

Boundary testing included submissions made on time, one minute late, one day late, one day and one minute late, two days late, three days late and more than three days late.

The expected penalties were checked against the system's assessment rules.

7.2 Integration Testing

Integration tests examined database persistence by saving student records and retrieving them through the SQLite repository.

The tests also checked the behaviour when searching for a username that does not exist.

7.3 Automated Test Results

The local CTest execution produced the following results:

Measurement	Result
Total automated tests	18
Passed	18
Failed	0
Pass rate	100%

The command used was:

```
ctest --test-dir build -C Debug --output-on-failure
```

Although all 18 automated tests passed, this does not mean that every system function has been tested.

The system testing plan contains 17 scenarios covering administration, enrolment, assignments, extensions, submissions, marking, moderation, access control, audit logging and backups. These scenarios remain recorded as not run.

The actual business-logic code coverage percentage also remains unconfirmed. Therefore, the 100% automated test pass rate should not be interpreted as 100% code coverage.

8. Identified Challenges and Gaps

Several challenges were encountered during development.

Role-based access control: Different users require different permissions. The system must prevent students from performing lecturer or administrator operations and ensure that only authorised HoDs can moderate and release results.

Assessment rules: Implementing late penalties, extensions and resubmissions requires careful handling of deadlines. The system must calculate penalties using the effective deadline and retain previous submission versions.

Database management: Persistent storage must maintain consistency across users, courses, assignments and submissions. Database errors must be handled without corrupting academic records.

Security: Academic information is sensitive. Password protection, access restrictions and audit logging are therefore important parts of the design.

The project also has several outstanding gaps:

1. The 17 planned system tests still need to be executed and documented.
2. The measured business-logic code coverage percentage needs to be confirmed.
3. A successful GitHub Actions run still needs verification.
4. The current architecture supports a single school computer rather than multiple computers accessing a shared server.
5. Further accessibility evaluation and user acceptance testing are required.

Future improvements should focus on completing system testing, verifying code coverage, confirming the CI workflow and conducting practical backup and restoration exercises.

The proposed whole-class deadline extension feature from the maintenance change request should also be considered for a future release, v1.1, after implementation and regression testing.

Conclusion

CampusNode LMS provides a foundation for managing academic activities within a school through a locally deployed learning management system. The application uses C++17, SQLite and libsodium to support user management, course administration, assignment submissions, assessment and controlled release of results.

The system implements four user roles and incorporates security measures, persistent storage and assessment business rules. The 18 successful automated tests provide evidence that selected business logic and database operations function as expected.

However, further work is required to complete system testing, confirm code coverage and verify continuous integration. Addressing these gaps will improve confidence in the system's reliability and readiness for deployment.

References

1. CS4433 Software Engineering Individual Project Assignment, 2026.
2. CampusNode LMS Software Requirements Specification, 2026.
3. CampusNode LMS Software Design and Behavioural Models, 2026.
4. CampusNode LMS Phase 5: Verification and Validation Report, 2026.
5. SQLite Documentation. https://www.sqlite.org/docs.html
6. Libsodium Documentation. https://doc.libsodium.org/